

```
# =====
# 1. SETUP AND DATA LOADING
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix

# Load the dataset
df = pd.read_csv('/content/drive/MyDrive/Invistico_Airline_Customer_Satisfaction.csv')
print(f"Dataset Shape: {df.shape}")
print(df.info())
```

```
Dataset Shape: (129880, 22)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
 #   Column                                     Non-Null Count  Dtype
---  -
 0   satisfaction                             129880 non-null  object
 1   Customer Type                             129880 non-null  object
 2   Age                                       129880 non-null  int64
 3   Type of Travel                           129880 non-null  object
 4   Class                                    129880 non-null  object
 5   Flight Distance                           129880 non-null  int64
 6   Seat comfort                             129880 non-null  int64
 7   Departure/Arrival time convenient         129880 non-null  int64
 8   Food and drink                           129880 non-null  int64
 9   Gate location                             129880 non-null  int64
10  Inflight wifi service                     129880 non-null  int64
11  Inflight entertainment                     129880 non-null  int64
12  Online support                             129880 non-null  int64
13  Ease of Online booking                     129880 non-null  int64
14  On-board service                           129880 non-null  int64
15  Leg room service                           129880 non-null  int64
16  Baggage handling                           129880 non-null  int64
17  Checkin service                           129880 non-null  int64
18  Cleanliness                               129880 non-null  int64
19  Online boarding                           129880 non-null  int64
20  Departure Delay in Minutes                 129880 non-null  int64
21  Arrival Delay in Minutes                   129487 non-null  float64
dtypes: float64(1), int64(17), object(4)
memory usage: 21.8+ MB
None
```

```
# =====
# 2. DATA PREPROCESSING & PIPELINE SETUP
```

```
# =====
# Handle missing values in target or features if any
# 'Arrival Delay in Minutes' often contains missing values; we'll impute with
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['A

# Separate Features (X) and Target (y)
X = df.drop(columns=['satisfaction'])
y = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0}) # Encode tar

# Identify column types
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
numeric_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()

print(f"\nCategorical columns: {categorical_cols}")
print(f"Numeric columns: {numeric_cols}")

# Create preprocessor using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_cols), # Scaling is vital for Logi
        ('cat', OneHotEncoder(drop='first'), categorical_cols) # drop='first'
    ])

# Split the data into Train and Test sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

```
Categorical columns: ['Customer Type', 'Type of Travel', 'Class']
Numeric columns: ['Age', 'Flight Distance', 'Seat comfort', 'Departure/Arriva
```

```
# =====
# 3. HYPERPARAMETER TUNING WITH GRIDSEARCHCV
# =====
# Build a pipeline with the preprocessor and Decision Tree
dt_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', DecisionTreeClassifier(random_state=42))
])

# Define the parameter grid to prevent overfitting
param_grid = {
    'classifier__max_depth': [3, 5, 7, 10, 15],
    'classifier__min_samples_split': [10, 20, 50],
    'classifier__min_samples_leaf': [5, 10, 20]
}

print("\nRunning GridSearchCV for Decision Tree...")
grid_search = GridSearchCV(dt_pipeline, param_grid, cv=5, scoring='f1', n_jo
grid_search.fit(X_train, y_train)

best_dt_model = grid_search.best_estimator_
```

```
# FIX: Changed from best_params__ to best_params_  
print(f"Best Hyperparameters: {grid_search.best_params_}")
```

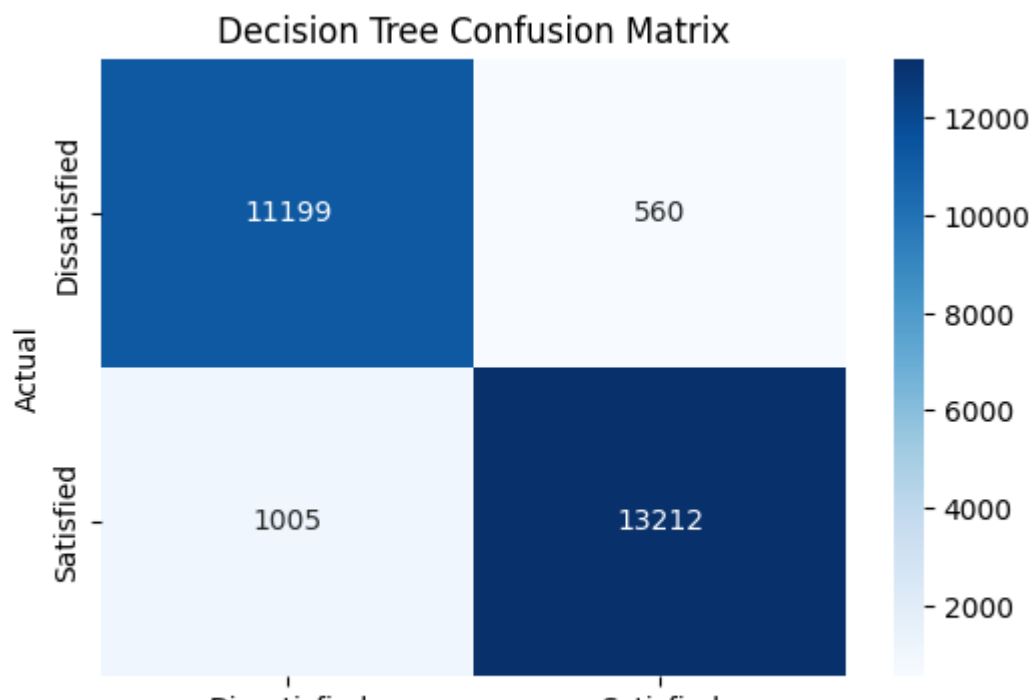
```
: 15, 'classifier__min_samples_leaf': 5, 'classifier__min_samples_split': 20}
```

```
# =====  
# 4. MODEL EVALUATION (DECISION TREE)  
# =====  
# Predictions  
y_pred_dt = best_dt_model.predict(X_test)  
  
# Metrics  
dt_f1 = f1_score(y_test, y_pred_dt)  
print(f"\nDecision Tree Test F1-Score: {dt_f1:.4f}")  
print("\nClassification Report (Decision Tree):")  
print(classification_report(y_test, y_pred_dt))  
  
# Confusion Matrix Plot  
cm_dt = confusion_matrix(y_test, y_pred_dt)  
plt.figure(figsize=(6, 4))  
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues', xticklabels=['Dissatis', 'Satisfis'])  
plt.title('Decision Tree Confusion Matrix')  
plt.ylabel('Actual')  
plt.xlabel('Predicted')  
plt.show()
```

Decision Tree Test F1-Score: 0.9441

Classification Report (Decision Tree):

	precision	recall	f1-score	support
0	0.92	0.95	0.93	11759
1	0.96	0.93	0.94	14217
accuracy			0.94	25976
macro avg	0.94	0.94	0.94	25976
weighted avg	0.94	0.94	0.94	25976



```
# =====
# 5. FEATURE IMPORTANCE EXTRACTION
# =====
# Get feature names after OneHotEncoding transformation
encoded_cat_features = best_dt_model.named_steps['preprocessor'].transformer
all_features = numeric_cols + encoded_cat_features

# Extract importances
importances = best_dt_model.named_steps['classifier'].feature_importances_

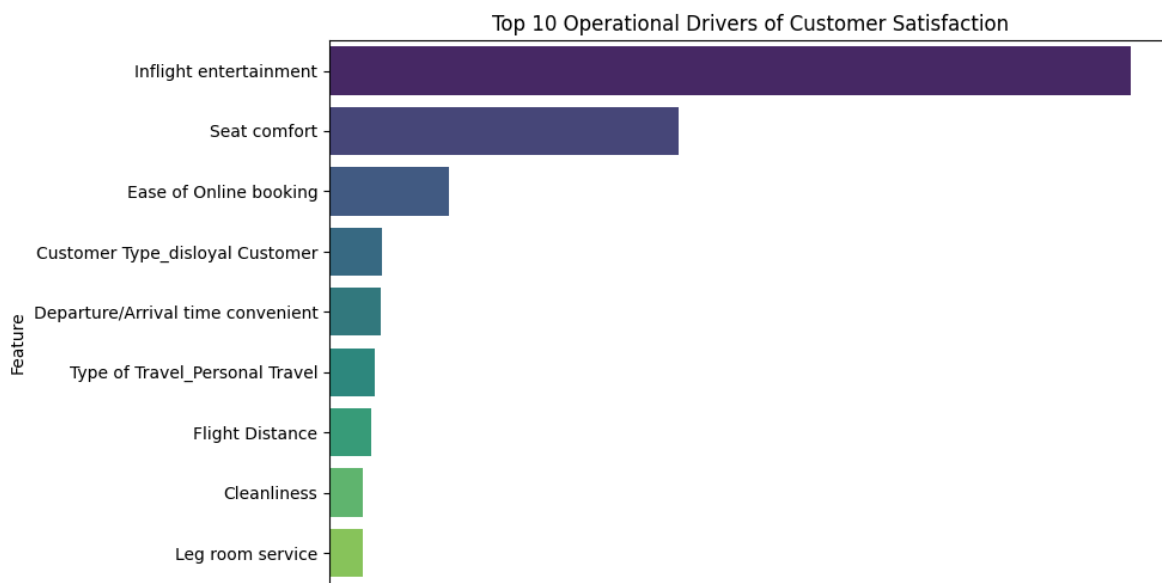
# Create a DataFrame for ranking
feature_imp_df = pd.DataFrame({'Feature': all_features, 'Importance': importances})

plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_imp_df.head(10), palette='magma')
plt.title('Top 10 Operational Drivers of Customer Satisfaction')
plt.xlabel('Importance Score')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_3569/3476338109.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in

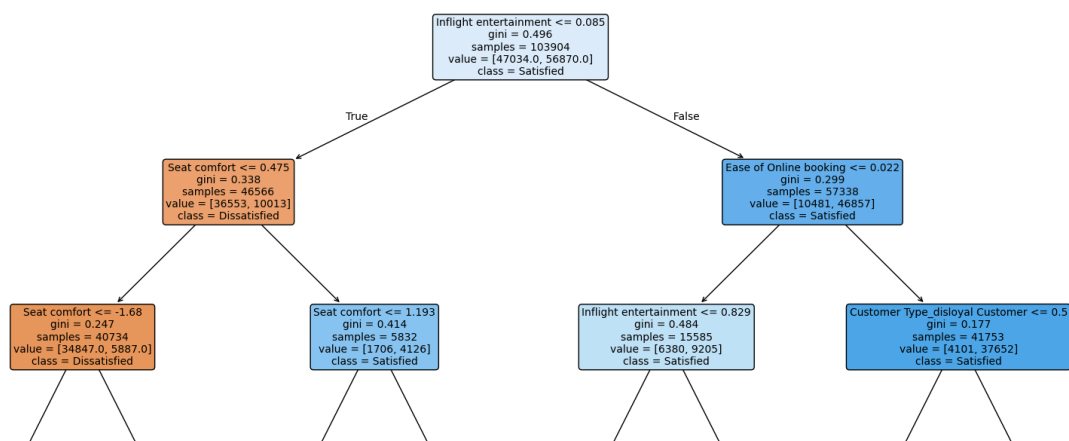
`sns.barplot(x='Importance', y='Feature', data=feature_imp_df.head(10), pale`



```
# =====
# 6. DECISION PATHWAY VISUALIZATION
# =====
# Note: Plotting a deep tree is illegible. We'll train a small shallow tree
plt.figure(figsize=(20, 10))
visual_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
visual_tree.fit(best_dt_model.named_steps['preprocessor'].transform(X_train))

plot_tree(visual_tree, feature_names=all_features, class_names=['Dissatisfied', 'Satisfied'])
plt.title("Decision Tree Pathway Logic (Truncated to Depth=3 for Clarity)",
plt.show()
```

Decision Tree Pathway Logic (Truncated to Depth=3 for Clarity)



```
# =====
# 7. COMPARISON WITH LOGISTIC REGRESSION
# =====
lr_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random
1)
```

```
lr_pipeline.fit(X_train, y_train)
y_pred_lr = lr_pipeline.predict(X_test)
lr_f1 = f1_score(y_test, y_pred_lr)

print("\n" + "="*40)
print(f"Logistic Regression Test F1-Score: {lr_f1:.4f}")
print("="*40)
print(classification_report(y_test, y_pred_lr))
```

```
=====
Logistic Regression Test F1-Score: 0.8437
=====
```